

Documentación Código - Problemática y Solución

Maycol Slon Buitrago Manrique

Oscar Mahecha Camargo

Ingeniería de sistemas, Corporación Universitaria Remington

Compiladores

2026

Tabla de contenido

Taller Depuración de Código con IA en Google Colab	3
1 Ejercicio 1: Análisis de Datos con Pandas	3
2 Ejercicio 2: Función Recursiva Incorrecta.....	4
3 Ejercicio 3: Gráfico de una Distribución con Matplotlib.....	5
4 Ejercicio 4: Web Scraping con BeautifulSoup.....	6
5 Ejercicio 5: Manipulación de Arrays con NumPy	7
6 Ejercicio 6: Uso de Hilos con threading	8
7 Ejercicio 7: API con requests	9
8 Ejercicio 8: Clase con Herencia en Python.....	10
9 Ejercicio 9: Conexión a Base de Datos SQLite	12
10 Ejercicio 10: Algoritmo de Ordenamiento (Bubble Sort)	13

Taller Depuración de Código con IA en Google Colab

En este documento se presenta la respectiva documentación de los ejercicios del taller de Google colab, se mostrará el error identificado y se incluirán imágenes para una representación mas clara de dichos errores. También se indicará la solución aplicada a cada uno de los ejercicios y se mostrará una alternativa que nos permita llegar a la solución más eficiente para a cada uno ellos. En esta sección también se hará uso de imágenes donde estará el código implementado.

1 Ejercicio 1: Análisis de Datos con Pandas

Problema: en el código se intenta leer un archivo CSV "ventas.csv" que no existe en el entorno de Colab, esto provoca un FileNotFoundError y detiene la ejecución.

```
import pandas as pd

# Cargar un archivo CSV
df = pd.read_csv("ventas.csv")

# Mostrar las primeras 5 filas
print(df.head())
```

Solución aplicada: se hace uso de un bloque try/except para envolver la lectura del archivo y capturar la excepción FileNotFoundError y en caso de que el archivo no exista, se informa al usuario y se genera un DataFrame de ejemplo para poder continuar trabajando sin que el programa se detenga.

```

# =====
# EJERCICIO 1: Análisis de Datos con Pandas
# ERROR: Intento de leer un archivo CSV que no existe (FileNotFoundError)
# =====
import pandas as pd

try:
    df = pd.read_csv("ventas.csv")
    print(df.head())
except FileNotFoundError:
    print("El archivo 'ventas.csv' no existe. Verifica la ruta o sube el archivo a Colab.")
    # Alternativa: crear datos de ejemplo para poder continuar probando el ejercicio
    df = pd.DataFrame({"producto": ["A", "B", "C"], "ventas": [10, 20, 30]})
    print("Usando datos de ejemplo:")
    print(df.head())

```

2 Ejercicio 2: Función Recursiva Incorrecta

Problema: no existe condición de parada en la función factorial, entonces esta se llama a sí misma indefinidamente hasta agotar la pila de llamadas (RecursionError). Esto es un error semántico.

```

def factorial(n):
    return n * factorial(n - 1)

print(factorial(5))

```

Solución aplicada: se agrega la condición base 'if n <= 1: return 1', esta es la que detiene la recursión cuando n llega a 1 o menos, permitiendo así, que las llamadas se resuelvan correctamente en cadena.

```

# =====
# EJERCICIO 2: Función Recursiva Incorrecta
# ERROR: Falta una condición de parada (caso base), causa RecursionError (recursi
# =====
def factorial(n):
    if n <= 1:          # caso base agregado
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120

```

... 120

3 Ejercicio 3: Gráfico de una Distribución con Matplotlib

Problema: el histograma se generaba sin parámetros explícitos, ejemplo, el número de bins, produciendo una visualización poco clara y confusa.

```

import numpy as np
import matplotlib.pyplot as plt

datos = np.random.normal(loc=50, scale=15, size=1000)

plt.hist(datos)
plt.show()

```

Solución aplicada: se hace la definición explícita del número de bins (bins=30), también se agregaron colores, título y etiquetas a los ejes, de esta manera aseguramos que la gráfica se renderice correctamente y se pueda entender fácilmente,

```
# =====
# EJERCICIO 3: Gráfico de una Distribución con Matplotlib
# ERROR: Uso incorrecto/incompleto de plt.hist(), no se especifican bins
#         ni se cierra la figura, lo que puede generar un gráfico vacío o mal formado
# =====
import numpy as np
import matplotlib.pyplot as plt

datos = np.random.normal(loc=50, scale=15, size=1000)

plt.figure(figsize=(8, 5))
plt.hist(datos, bins=30, color="steelblue", edgecolor="black") # bins definido explícita
plt.title("Distribución de datos")
plt.xlabel("Valor")
plt.ylabel("Frecuencia")
plt.show()
```

4 Ejercicio 4: Web Scraping con BeautifulSoup

Problema: la solicitud `requests.get(url)` se ejecuta sin un encabezado de User-Agent, es por ellos que algunos servidores rechazan la petición o devuelven una respuesta distinta a la esperada.

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text)
```

Solución aplicada: agregamos un diccionario `headers` con un User-Agent válido y se pasó como parámetro a `requests.get()`, simulando así una solicitud realizada desde un navegador real.

```
# =====
# EJERCICIO 4: Web Scraping con BeautifulSoup
# ERROR: La página rechaza requests.get() por no incluir un encabezado User-Agent
# =====
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"} # encabezado

response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text if soup.title else "No se encontró el título")

Example Domain
```

5 Ejercicio 5: Manipulación de Arrays con NumPy

Problema: el arreglo se genera con `np.arange(10)`, entonces produce 10 elementos, el error es cuando se intenta convertir en una matriz de 3x3, con `reshape(3,3)`, esto genera un `ValueError` por incompatibilidad de dimensiones. Este es un error semántico.

```
import numpy as np

array = np.arange(10)
matriz = array.reshape(3, 3)
print(matriz)
```

Solución aplicada: ajustamos la generación del arreglo a `np.arange(9)`, de modo que el número total de elementos que son 9 coincida exactamente con las dimensiones solicitadas en el `reshape 3x3`.

```
import numpy as np

array = np.arange(9) # Se cambió a 9 elementos para que sea compatible con 3x3
matriz = array.reshape(3, 3)
print(matriz)

# Opcional: Si necesitas el array de 10 elementos, podrías cambiar la forma a una
# array_10 = np.arange(10)
# matriz_10 = array_10.reshape(2, 5) # 0 (5, 2)
# print(matriz_10)

[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

6 Ejercicio 6: Uso de Hilos con threading

Problema: dos hilos incrementan una variable compartida (contador), pero, no tienen ningún medio de sincronización, esto provoca una condición de carrera (race condition) y resultados inconsistentes.

```
import threading

contador = 0

def incrementar():
    global contador
    for _ in range(1000):
        contador += 1

t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=incrementar)

t1.start()
t2.start()
t1.join()
t2.join()

print("Valor final del contador:", contador)
```

Solución aplicada: se hace uso de un `threading.Lock()`, este protege la sección crítica del código, que es el incremento del contador, todo ello mediante `'with lock:'`, garantizando que solo un hilo acceda a la variable a la vez.

```
# =====
# EJERCICIO 6: Uso de Hilos con threading
# ERROR: Falta sincronización (Lock) entre hilos, causando condición de carrera (race condition)
# =====
import threading

contador = 0
lock = threading.Lock()      # candado agregado para sincronizar el acceso

def incrementar():
    global contador
    for _ in range(1000):
        with lock:            # sección crítica protegida
            contador += 1

t1 = threading.Thread(target=incrementar)
t2 = threading.Thread(target=incrementar)

t1.start()
t2.start()
t1.join()
t2.join()

print("Valor final del contador:", contador) # siempre 2000
```

7 Ejercicio 7: API con requests

Problema: el código no está configurado para que maneje errores HTTP, entonces si la API no responde correctamente, ejemplo, error 404 o 500, o problemas de conexión, el programa puede fallar sin dar ninguna información útil.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)

print(response.json())
```

Solución aplicada: se envuelve la solicitud en un bloque try/except, también se agregó un timeout y se usó `response.raise_for_status()` para detectar y manejar de forma explícita errores de la petición HTTP.

```
# =====
# EJERCICIO 7: API con requests
# ERROR: No maneja errores HTTP (por ejemplo, si la URL falla o responde con error)
# =====
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

try:
    response = requests.get(url, timeout=5)
    response.raise_for_status() # lanza excepción si el código HTTP indica error
    print(response.json())
except requests.exceptions.RequestException as e:
    print(f"Error al consultar la API: {e}")

... {'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi op
```

8 Ejercicio 8: Clase con Herencia en Python

Problema: la clase Perro hereda de Animal, pero su método `init` no llama al constructor de la clase base, entonces el atributo `nombre` nunca podrá asignarse provocando que el programa falle al usarse

```

class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

class Perro(Animal):
    def __init__(self, nombre, raza):
        self.raza = raza

    def ladrar(self):
        print(f"{self.nombre} está ladrando.")

mi_perro = Perro("Firulais", "Labrador")
mi_perro.ladrar()

```

... -----
AttributeError Traceback (most recent call last)
/tmp/ipykernel_2294/1856468180.py in <cell line: 0>()

Solución aplicada: se agrega la llamada `super().__init__(nombre)` dentro del constructor de `Perro`, para que así, se ejecute correctamente la inicialización heredada de la clase `Animal`.

```

# =====
# EJERCICIO 8: Clase con Herencia en Python
# ERROR: Perro.__init__ no llama a Animal.__init__, por lo que 'nombre' nunca se asigna
# =====
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

class Perro(Animal):
    def __init__(self, nombre, raza):
        super().__init__(nombre) # se inicializa la clase base
        self.raza = raza

    def ladrar(self):
        print(f"{self.nombre} está ladrando.")

mi_perro = Perro("Firulais", "Labrador")
mi_perro.ladrar()

```

... Firulais está ladrando.

9 Ejercicio 9: Conexión a Base de Datos SQLite

Problema: cuando se ejecutan las sentencias SQL de creación de tabla e inserción no se está llamando a `conn.commit()`, es por ello que los cambios no se guardan de forma permanente en la base de datos comprometiendo la integridad de la información.

```
import sqlite3

conn = sqlite3.connect("database.db")
cursor = conn.cursor()

cursor.execute("CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY, nombre TEXT)")
cursor.execute("INSERT INTO usuarios (nombre) VALUES ('Juan')")

conn.close()
```

Solución aplicada: se agrega el `conn.commit()` antes de cerrar la conexión, así aseguramos que las transacciones, es decir, creación de tabla e inserción de datos puedan confirmarse y se guarden en el archivo `database.db`.

```
import sqlite3

conn = sqlite3.connect("database.db")
cursor = conn.cursor()

cursor.execute("CREATE TABLE IF NOT EXISTS usuarios (id INTEGER PRIMARY KEY, nombre TEXT)")
cursor.execute("INSERT INTO usuarios (nombre) VALUES ('Juan')")

conn.commit()      # confirmación de cambios agregada
conn.close()
```

10 Ejercicio 10: Algoritmo de Ordenamiento (Bubble Sort)

Problema: el bucle interno usa siempre `range(n-1)` y no se reduce en cada pasada externa, esto hace que el algoritmo repita comparaciones que no son necesarias sobre elementos que ya están ordenados, esto es ineficiencia pura, no un error de lógica que impida el proceso de ordenamiento.

```
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        for j in range(n - 1):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]

numeros = [5, 3, 8, 1, 2]
bubble_sort(numeros)
print(numeros)
```

Solución: se ajusta el rango del bucle interno a `range(0, n-i-1)` para que en cada pasada se reduzca la cantidad de comparaciones, puesto que los últimos elementos ya quedaron ordenados.

```
# =====
# EJERCICIO 10: Algoritmo de Ordenamiento (Bubble Sort)
# ERROR: El rango interno no se reduce con cada pasada externa, lo que hace el
#         algoritmo ineficiente (recorre comparaciones ya ordenadas innecesariamente)
# =====
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        for j in range(0, n - i - 1): # el rango se reduce en cada pasada
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]

numeros = [5, 3, 8, 1, 2]
bubble_sort(numeros)
print(numeros) # [1, 2, 3, 5, 8]
```

... [1, 2, 3, 5, 8]